DOC GENIE

Naan Mudhalvan Project Report Submitted to **Manonmaniam Sundaranar University** in The Partial fulfilment of the Requirement for the award of the Degree of **Bachelor of Computer Science**

Submitted By

TEAM ID - SWTID-2026-3117

M.A.K.MARIYAM MASKOORA – 23082241802112022(TEAM LEADER)

P.A.MOHODOOM FATHIMA FARHANA – 23082241802112023(TEAM MEMBER)

S.MUGESHWARI – 23082241802112024 (TEAM MEMBER)

C.MUTHU KALYANI –23082241802112025 (TEAM MEMBER)

DEPARTMENT OF COMPUTER SCIENCE

WAVOO WAJEEHA WOMEN'S COLLEGE OF ARTS & SCIENCE

KAYALPATNAM – 628 204

APRIL – 2026

CERTIFICATE

WAVOO WAJEEHA WOMEN'S COLLEGE OF ARTS AND SCIENCE

KAYALPATNAM – 628 204

CERTIFICATE

This is to certify that project entitled as

" DOC GENIE "

Is a Bonafide work done by

TEAM ID - SWTID-2026-3117

M.A.K.MARIYAM MASKOORA – 23082241802112022(TEAM LEADER)

P.A.MOHODOOM FATHIMA FARHANA – 23082241802112023(TEAM MEMBER)

S.MUGESHWARI – 23082241802112024 (TEAM MEMBER)

C.MUTHU KALYANI –23082241802112025 (TEAM MEMBER)

In partial fulfillment of the requirement for B.Sc. Degree Course in Computer Science (2025-2026) of the Manonmaniam Sundaranar University for NaanMudhalvan Course(Agentic AI Application Developer) is done .

Signature of the Staff Incharge

Signature of the H.O.D

EXTERNAL EXAMINERS

1.

2.

DECLARATION

DECLARATION

This project is submitted for the Naan Mudhalvan Course Agentic AI Application Developer

Subject code _____and the external examination is conducted on _____.

.

Signature of the Candidates

1. **M.A.K.MARIYAM MASKOORA**

2. **P.A.MOHODOOM FATHIMA FARHANA**

3. **S.MUGESHWARI**

4. **C.MUTHU KALYANI**

DATE :

PLACE: Kayalpatnam

CONTENT

TABLE OF CONTENT

S.NO	TOPIC	PAGE NO
1.	PROJECT DESCRIPTION	1
2.	SCENARIOS	3
3.	ARCHITECTURE OVERVIEW	6
4.	PROJECT WORKFLOW	9
5.	PROJECT STRUCTURE	13
6.	IMPLEMENTATION PHASE	15
7.	SCREENSHOTS	35
8.	FUTURE ENHANCEMENT	38
9.	CONCLUSION	40

PROJECT DESCRIPTION

1.PROJECT DESCRIPTION

Doc-Genie is an intelligent Python documentation platform that automatically generates professional docstrings for Python functions. The system uses **Abstract Syntax Tree (AST) analysis** and advanced code understanding techniques to create accurate and consistent documentation through an interactive **Gradio-based interface**. It helps developers maintain clear and well-structured documentation without writing docstrings manually.

The platform analyzes Python function code and generates documentation in standard formats such as **Google style** and **NumPy style**. By examining function definitions, parameters, return values, and internal logic, the system produces structured documentation that clearly explains the functionality of each Python function.

Using advanced **AST parsing algorithms** and Python introspection capabilities, the system extracts function signatures, identifies parameters and return types, and analyzes the internal logic of the code. Based on this analysis, Doc-Genie generates context-aware docstrings that improve code readability, maintainability, and consistency.

In addition, Doc-Genie provides **export features** that allow users to download generated documentation in **PDF and TXT formats**. The system also includes sharing options for **WhatsApp and Facebook**, making it easier to share documentation with team members. With its user-friendly interface and automated process, Doc-Genie simplifies Python documentation and improves development efficiency.

SCENARIOS

2.SCENARIOS

Scenario 1: Existing Python Code Documentation

A developer receives an existing Python project where many functions do not contain proper documentation. Instead of manually writing docstrings for every function, the developer uses **Doc-Genie** to upload the Python file through the Gradio interface. The system analyzes the function using **AST parsing**, extracts parameters and return values, and automatically generates **Google-style docstrings**. This helps the developer quickly create accurate and consistent documentation for the entire code file.

Scenario 2: Python Function Development

While developing new Python functions, developers often forget to write proper documentation. Using **Doc-Genie**, they can paste their function code into the interface and generate **Google or NumPy style docstrings** instantly. The tool analyzes the function structure, detects parameters and operations, and produces meaningful documentation that explains the function's behavior. This ensures that every function created during development is properly documented.

. Scenario 3: Team Documentation Standardization

In a development team, different programmers may write documentation in different styles. Doc-Genie helps teams maintain **consistent documentation standards** across their Python projects. The system automatically generates structured docstrings for each function and allows users to export documentation as **PDF or TXT files**. This makes it easier for teams to maintain uniform documentation and review project documentation when needed**.**

Scenario 4: Learning Python Documentation

Doc-Genie can also be used for educational purposes. Students learning Python programming can input their functions into the system and see how professional **docstrings are generated automatically**. This helps them understand the structure of proper documentation and learn how functions should be explained clearly in real-world projects.

Scenario 5: Code Review and Documentation Check

During code review, developers can use **Doc-Genie** to verify whether functions are properly documented. The system generates expected docstrings based on function logic and parameters. Reviewers can compare the generated documentation with the existing documentation to identify missing or incomplete information. This helps improve **code readability and documentation quality** in software projects.

ARCHITECTURE OVERVIEW

3. ARCHITECTURE OVERVIEW

Doc-Genie is developed as a **modular Python-based system** that combines Python's **AST (Abstract Syntax Tree) module** with the **Gradio framework** to automatically generate documentation for Python functions. The architecture focuses on accurate code analysis, automatic docstring generation, and a simple user interface.

The system analyzes Python function code using **AST parsing** to extract details such as function name, parameters, and return values. Based on this analysis, it generates structured docstrings in different styles and provides options to export the generated documentation.

Core Technologies

- **Python 3.8+** – Core programming language used for system development
- **Gradio 4.0+** – Interactive interface for code input and documentation generation
- **AST (Abstract Syntax Tree)** – Used for analyzing Python function structure
- **ReportLab** – Used for generating PDF documentation
- **Regular Expressions (Regex)** – Used for pattern matching in code analysis
- **Google Colab** – Cloud environment for development and execution
- **HTML5 / CSS3** – Used for interface styling

Pre-requisites

Software Requirements

- **Python 3.8+**: Download from *python.org* (required for AST-based code analysis)
- **Google Account**: Needed to access and run the project in **Google Colab**
- **pip**: Python package manager used to install required libraries
- **Web Browser**: Chrome, Firefox, Safari, or Microsoft Edge
- **Internet Connection**: Required for running Colab and accessing Gradio sharing links

Knowledge Prerequisites

- **Python Fundamentals** – Basic understanding of functions, decorators, and type hints
- **Python AST** – Knowledge of Abstract Syntax Trees and code parsing
- **Python Docstrings** – Familiarity with Google and NumPy docstring formats
- **Gradio Framework** – Understanding of interface components and callbacks
- **Code Analysis** – Basic idea of static code analysis concepts
- **Google Colab** – Basic knowledge of notebook interface and cell execution

Hardware Requirements

- **Processor** – Any modern processor (Intel i3 or equivalent)
- **RAM** – Minimum 2GB (Colab provides additional cloud resources)
- **Storage** – Around 500MB free space for application and generated files
- **Display** – Minimum 1024 × 768 resolution for proper interface usability

PROJECT WORKFLOW

4.PROJECT WORKFLOW

Phase 1: Environment Setup and Core Architecture

Activity 1.1: Install Dependencies and Configure Environment

- Create a **requirements.txt** file including all required libraries.
- Set up the **Google Colab environment** with necessary configurations.
- Install required packages such as **gradio, transformers, torch, reportlab, and requests.**
- Verify that all packages are installed and confirm the **Python AST module**.

Activity 1.2: Design DocGenieAnalyzer Core Class

- Develop the **DocGenieAnalyzer class** for performing code analysis.
- Design the **AST parsing mechanism** for extracting Python functions.
- Implement detection of **type annotations and parameters.**
- Set up a **generation history tracking system** to store documentation results.

Phase 2: Code Analysis Engine

Activity 2.1: Implement Function Signature Extraction

- Create the **extract_function_signature() method.**
- Parse the **function name** from the AST structure.
- Extract **parameter details** including names and types.
- Detect **default parameter values.**
- Identify **return type annotations.**

Activity 2.2: Build Function Logic Analysis System

- Create the **analyze_function_logic() method.**
- Detect **loops** such as *for* and *while* in the function body.
- Identify **conditional statements** like *if* and *else*.
- Detect **mathematical operations** (addition, multiplication, division, subtraction).
- Track **return statements and return values.**
- Identify **function calls and variables used.**
- Generate **descriptive summaries** based on analysis results.

Phase 3: Interface Development and User Experience

Activity 3.1: Develop Code Input and Output Interface

- Create a **code input editor** with syntax highlighting
- Implement **file upload support for .py files**
- Add **docstring style selection** (Google / NumPy)
- Display the **generated output code** with syntax highlighting
- Add a **Generate button** to start the documentation process

Activity 3.2: Implement Export Options

- Create **PDF export functionality using ReportLab**
- Implement **TXT export for generated documentation**
- Enable **file download capability**
- Display **export status messages** for users

Phase 4: Export and Reporting System

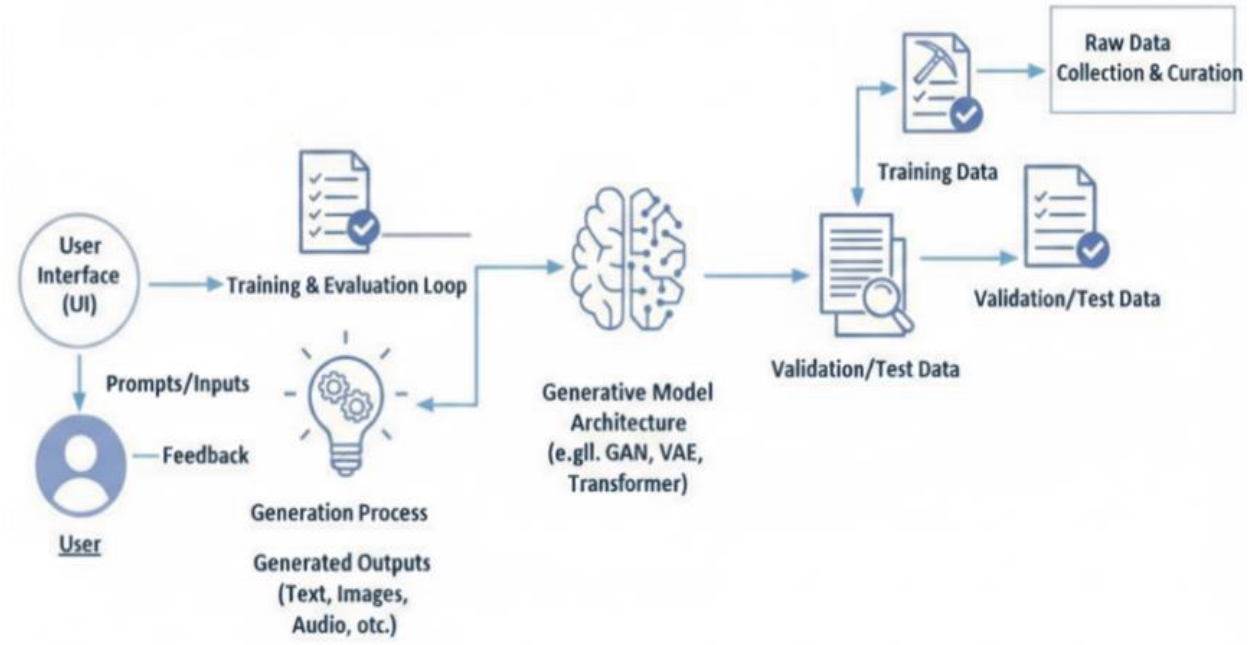
Activity 4.1: Implement PDF Report Generation

- Create the **ReportLab document structure** for PDF output
- Design a **professional layout for documentation reports**
- Format **code blocks clearly** within the PDF
- Add **pagination for multiple documentation entries**
- Include **metadata and timestamps**

Activity 4.2: Implement Text File Export

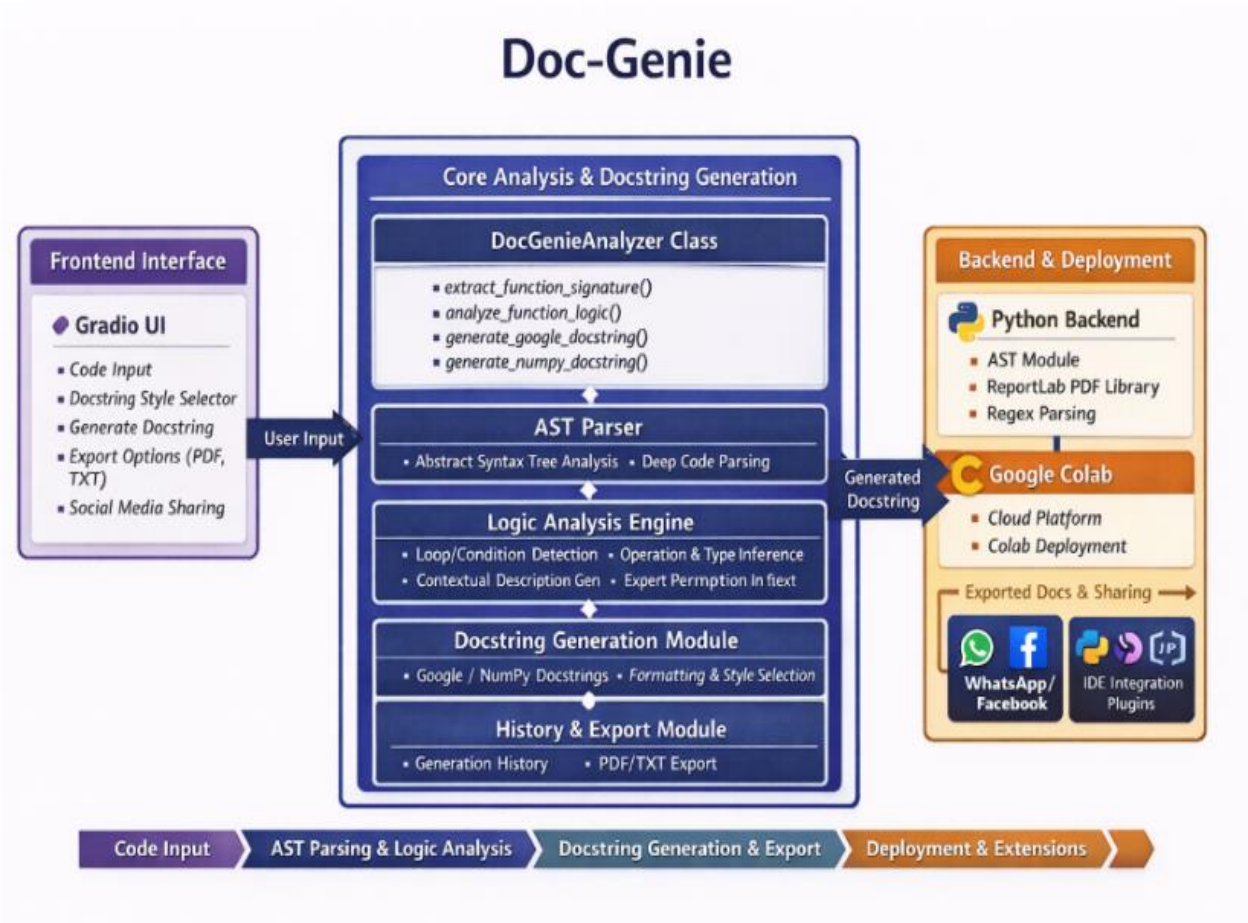
- Generate **formatted text documentation output**
- Add **clear section separators for readability**
- Include **metadata and analysis information**
- Support **exporting multiple documentation results in one file**

TECHNICAL STRUCTURE



PROJECT STRUCTURE

5.PROJECT STRUCTURE



IMPLEMENTATION PHASE

6.IMPLEMENTATION PHASE

Activity 1.1:Install Dependencies

"""DOC Genie.py

Automatically generated by Colab.

Original file is located at

https://colab.research.google.com/drive/1P7u-oYGindv2t3pza3ECrT4AuUqlLhWe

"""

!pip install -U transformers

"""## Local Inference on GPU

Model page: https://huggingface.co/ibm-granite/granite-3.3-2b-instruct

⚠ If the generated code snippets do not work, please open an issue on either the [model repo](https://huggingface.co/ibm-granite/granite-3.3-2b-instruct)and/or on [huggingface.js](https://github.com/huggingface/huggingface.js/blob/main/packages/tasks/src/model-libraries-snippets.ts)

"""

1. Use a pipeline as a high-level helper

```
from transformers import pipeline

pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")

messages = [

    {"role": "user", "content": "Who are you?"},

]

pipe(messages)
```

2. Load model directly

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")

model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")

messages = [

    {"role": "user", "content": "Who are you?"},

]

inputs = tokenizer.apply_chat_template(

messages,

add_generation_prompt=True,

tokenize=True,

return_dict=True,

return_tensors="pt",

).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)

print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))
```

!pip install -q gradio transformers torch black autopep8 reportlab requests

!pip install -q gradio transformers torch black autopep8 reportlab requests ast-super-parser

```
import gradio as gr

from datetime import datetime

from pathlib import Path
```

```
import json
import re
import ast
from reportlab.lib.pagesizes import letter, A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak, Table, TableStyle, Preformatted
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib import colors
import requests
import urllib.parse
import inspect
```

Activity 1.1: DOC-GENIE CORE LOGIC - IMPROVED VERSION

```
class DocGenieAnalyzer:
    """Analyze Python functions and generate professional docstrings"""
    @staticmethod
    def extract_function_signature(code):
        """Extract function name, parameters, return type from code"""
        try:
            tree = ast.parse(code)
            func_def = tree.body[0]
            if not isinstance(func_def, ast.FunctionDef):
                return None, None
```

```
    func_name = func_def.name
    params = []
    for arg in func_def.args.args:
        param_type = "Any"
        if arg.annotation:
            param_type = ast.unparse(arg.annotation) if hasattr(ast, 'unparse') else "Any"
        params.append({
            'name': arg.arg,
            'type': param_type,
            'default': None
        })
    for i, default in enumerate(func_def.args.defaults):
        idx = len(params) - len(func_def.args.defaults) + i
        if idx >= 0:
            params[idx]['default'] = ast.unparse(default) if hasattr(ast, 'unparse') else
str(default)
    return_type = "Any"
    if func_def.returns:
        return_type = ast.unparse(func_def.returns) if hasattr(ast, 'unparse') else "Any"
    return {
        'name': func_name,
        'params': params,
        'return_type': return_type
```

```
        }, func_def
    except:
        return None, None
@staticmethod
def analyze_function_logic(func_def, code):
    """Deep analysis of function body to understand actual behavior"""
    try:
        body = func_def.body
        analysis = {
            'has_loops': False,
            'has_conditions': False,
            'has_return': False,
            'loop_types': [],
            'condition_types': [],
            'operations': [],
            'return_types': [],
            'variables_used': set(),
            'function_calls': [],
            'description': ""
        }
        # Walk through AST
        for node in ast.walk(func_def):
            if isinstance(node, ast.For):
                analysis['has_loops'] = True
```

```
            analysis['loop_types'].append('for')
        elif isinstance(node, ast.While):
            analysis['has_loops'] = True
            analysis['loop_types'].append('while')
        elif isinstance(node, ast.If):
            analysis['has_conditions'] = True
            analysis['condition_types'].append('if')
        elif isinstance(node, ast.Return):
            analysis['has_return'] = True
            if node.value:
                analysis['return_types'].append(ast.unparse(node.value) if hasattr(ast,
'unparse') else 'value')
        elif isinstance(node, ast.BinOp):
            op = node.op
            if isinstance(op, ast.Mult):
                analysis['operations'].append('multiplication')
            elif isinstance(op, ast.Add):
                analysis['operations'].append('addition')
            elif isinstance(op, ast.Sub):
                analysis['operations'].append('subtraction')
            elif isinstance(op, ast.Div):
                analysis['operations'].append('division')
        elif isinstance(node, ast.Call):
            if isinstance(node.func, ast.Name):
```

```
            analysis['function_calls'].append(node.func.id)
    elif isinstance(node, ast.Name):
        analysis['variables_used'].add(node.id)
# Build detailed description
description_parts = []
if func_def.name:
    description_parts.append(f"Executes {func_def.name} operation")
if analysis['has_conditions']:
    description_parts.append("with conditional logic")
if analysis['has_loops']:
    description_parts.append("and iterates through values")
if 'multiplication' in analysis['operations'] and 'subtraction' in analysis['operations']:
    description_parts.append("performing mathematical calculations")
elif 'multiplication' in analysis['operations']:
    description_parts.append("using multiplication operations")
elif 'addition' in analysis['operations']:
    description_parts.append("using addition operations")
elif 'division' in analysis['operations']:
    description_parts.append("using division operations")
if analysis['return_types']:
    description_parts.append(f"and returns the computed result")
analysis['description'] = " ".join(description_parts) + "."
if len(analysis['description']) < 20:
    analysis['description'] = "Performs computation based on input parameters."
```

```
        return analysis
    except Exception as e:
        return {
            'has_loops': False,
            'has_conditions': False,
            'has_return': False,
            'description': "Executes function logic based on provided parameters."
        }
@staticmethod
def generate_google_docstring(signature, analysis, code):
    """Generate accurate Google-style docstring"""
    summary = analysis.get('description', 'Processes input and returns result.')
    docstring = f'    """{summary}\n'
    if signature['params']:
        docstring += '\n    Args:\n'
        for param in signature['params']:
            default_str = f" (default: {param['default']})" if param['default'] else ""
            docstring += f"        {param['name']} ({param['type']}): Parameter for controlling
{param['name']} behavior{default_str}.\n"
    docstring += f"\n    Returns:\n        {signature['return_type']}: The result of the
computation.\n"
    docstring += '    """\n'
    return docstring
```

```
    @staticmethod
    def generate_numpy_docstring(signature, analysis, code):
        """Generate accurate NumPy-style docstring"""
        summary = analysis.get('description', 'Processes input and returns result.')
        docstring = f'    """{summary}\n'
        if signature['params']:
            docstring += '\n    Parameters\n    ----------\n'
            for param in signature['params']:
                default_str = f", default: {param['default']}" if param['default'] else ""
                docstring += f"    {param['name']} : {param['type']}{default_str}\n"
                docstring += f"        Parameter for controlling {param['name']} behavior.\n"
        docstring += f"\n    Returns\n    -------\n    {signature['return_type']}\n"
        docstring += f"        The result of the computation.\n"
        docstring += '    """\n'
        return docstring
analyzer = DocGenieAnalyzer()
generation_history = []
def generate_docstring(code, style="google"):
    """Generate docstring for Python function with accurate analysis"""
    if not code.strip():
        return None, "✕ Empty code provided!"
    try:
        signature, func_def = analyzer.extract_function_signature(code)
        if not signature:
```

```
        return None, "❌ Invalid Python function code!"
    # Deep analysis of the function
    analysis = analyzer.analyze_function_logic(func_def, code)
    if style.lower() == "google":
        docstring = analyzer.generate_google_docstring(signature, analysis, code)
    else:
        docstring = analyzer.generate_numpy_docstring(signature, analysis, code)
    # Extract function definition line
    code_lines = code.split('\n')
    func_line = code_lines[0]
    # Build final function with docstring
    final_code = func_line + '\n' + docstring
    for line in code_lines[1:]:
        final_code += line + '\n'
    generation_history.append({
        'code': code,
        'docstring': docstring,
        'style': style,
        'timestamp': datetime.now().isoformat(),
        'analysis': analysis
    })
    return final_code, f"✅ {style.upper()} Docstring Generated!"
except Exception as e:
    return None, f"❌ Error: {str(e)}"
```

```
def process_upload(file):
    """Process uploaded Python file"""
    if not file:
        return None, "❌ No file uploaded!"
    try:
        content = file.read().decode('utf-8')
        return content, "✅ File loaded successfully!"
    except:
        return None, "❌ Failed to read file!"
def clear_history():
    """Clear generation history"""
    global generation_history
    generation_history = []
    return "✅ History cleared!"
def download_pdf():
    """Download all generated docstrings as PDF"""
    if not generation_history:
        return "❌ No documentation generated!", None
    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.pdf"
    try:
        doc = SimpleDocTemplate(filename, pagesize=A4, topMargin=0.5*inch,
bottomMargin=0.5*inch)
```

```
    styles = getSampleStyleSheet()
    story = []
    title_style = ParagraphStyle(
        'Title',
        parent=styles['Heading1'],
        fontSize=24,
        textColor='#FF6B6B',
        spaceAfter=20,
        alignment=1
    )
    code_style = ParagraphStyle(
        'Code',
        parent=styles['Normal'],
        fontSize=9,
        fontName='Courier',
        backColor='#F0F0F0',
        borderColor='#CCCCCC',
        borderWidth=1,
        borderPadding=6
    )
    story.append(Paragraph("📚 Doc-Genie Documentation Report", title_style))
    story.append(Paragraph(f"Generated: {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}", styles['Normal']))
    story.append(Spacer(1, 0.3*inch))
```

```
        for i, gen in enumerate(generation_history, 1):
            story.append(Paragraph(f"<b>Documentation #{i}</b>", styles['Heading2']))
            story.append(Paragraph(f"<b>Style:</b> {gen['style'].upper()}", styles['Normal']))
            story.append(Spacer(1, 0.1*inch))
            story.append(Paragraph("<b>Original Code:</b>", styles['Normal']))
            story.append(Preformatted(gen['code'], code_style))
            story.append(Spacer(1, 0.15*inch))
            story.append(Paragraph("<b>Generated Docstring:</b>", styles['Normal']))
            story.append(Preformatted(gen['docstring'], code_style))
            story.append(Spacer(1, 0.3*inch))
            if i % 2 == 0 and i != len(generation_history):
                story.append(PageBreak())
        story.append(Spacer(1, 0.3*inch))
        story.append(Paragraph(f"Total Documentations: {len(generation_history)}",
styles['Normal']))
        doc.build(story)
        return f"✅ PDF Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None
def download_txt():
    """Download all generated docstrings as TXT"""
    if not generation_history:
        return "❌ No documentation generated!", None
```

```
    content = f"Doc-Genie Documentation Report\nGenerated:
{datetime.now()}\n{'='*80}\n\n"
    for i, gen in enumerate(generation_history, 1):
        content += f"Documentation #{i}\n"
        content += f"Style: {gen['style'].upper()}\n"
        content += f"Generated: {gen['timestamp']}\n"
        content += f"{'-'*80}\n\n"
        content += f"ORIGINAL CODE:\n{gen['code']}\n\n"
        content += f"GENERATED DOCSTRING:\n{gen['docstring']}\n\n"
        content += f"{'-'*80}\n\n"
    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.txt"
    try:
        Path(filename).write_text(content)
        return f"✅ TXT Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None
def generate_whatsapp_link():
    """Generate WhatsApp share link"""
    if not generation_history:
        return "❌ No documentation to share!"
    message = "📚 *Doc-Genie Documentation*\n\n"
    for i, gen in enumerate(generation_history[-3:], 1):
        message += f"*Doc {i}:*\n"
```

```
        message += f"Style: {gen['style']}\n"
        message += f"`{gen['code'][:60]}...`\n\n"
    encoded = urllib.parse.quote(message)
    link = f"https://wa.me/?text={encoded}"
    return f"✅ WhatsApp Link:\n\n{link}\n\n📱 Share with team!"
def generate_facebook_link():
    """Generate Facebook share link"""
    if not generation_history:
        return "❌ No documentation to share!"
    message = f"I generated {len(generation_history)} professional Python function docstrings with Doc-Genie! Check it out!"
    encoded = urllib.parse.quote(message)
    link = f"https://www.facebook.com/sharer/sharer.php?quote={encoded}"
    return f"✅ Facebook Link:\n\n{link}\n\n👍 Share on Facebook!"
print("🚀 Loading Doc-Genie Improved Version...")
with gr.Blocks(
    title="Doc-Genie - Python Documentation Generator",
    theme=gr.themes.Soft(),
    css="""
    body { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .gradio-container { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .button-primary { background: linear-gradient(135deg, #FF6B6B 0%, #FF8E72 100%); }
    .button-secondary { background: linear-gradient(135deg, #4ECDC4 0%, #44A08D 100%); }
```

```
    .code-block { background: #0f3460; border: 2px solid #4ECDC4; border-radius: 8px;
padding: 15px; }
    """
) as demo:
    gr.Markdown("""
    # 📚 Doc-Genie v2.0
    ## Professional Python Function Docstring Generator
    Transform your Python functions with AI-powered, accurate documentation!
    """)
    with gr.Row():
        with gr.Column(scale=3):
            # Code Input Area
            gr.Markdown("### 📝 Input Function Code")
            code_input = gr.Code(
                language="python",
                label="Python Function",
                lines=10,
                value="def calculate_factorial(n):\n    if n == 0:\n        return 1\n    return n *
calculate_factorial(n-1)"
            )
            # File Upload
            file_upload = gr.File(label="📤 Or Upload .py File", file_types=['.py'])
            upload_status = gr.Textbox(label="Upload Status", interactive=False, lines=1)
```

```
def process_and_display(file):
    if file:
        content, status = process_upload(file)
        return content, status
    return "", "No file"
file_upload.change(process_and_display, inputs=[file_upload], outputs=[code_input, upload_status])
# Style Selection
with gr.Row():
    docstring_style = gr.Radio(["google", "numpy"], value="google", label="🎨 Docstring Style")
    generate_btn = gr.Button("✨ Generate Docstring", size="lg", variant="primary", scale=2)
# Output Area
gr.Markdown("### 📄 Generated Output")
code_output = gr.Code(
    language="python",
    label="Function with Docstring",
    lines=15,
    interactive=False
)
gen_status = gr.Textbox(label="Status", interactive=False, lines=1)
generate_btn.click(
    generate_docstring,
    inputs=[code_input, docstring_style],
```

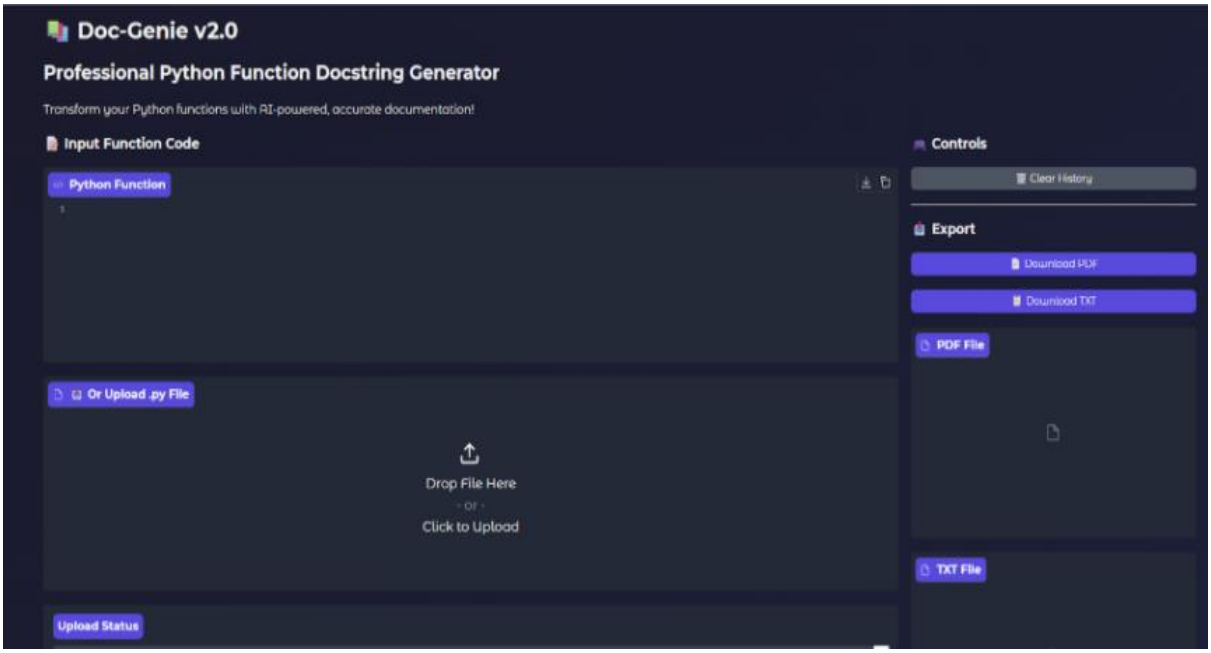
```
        outputs=[code_output, gen_status]
    )
with gr.Column(scale=1):
    gr.Markdown("### 🎮 Controls")
    clear_btn = gr.Button("🗑️ Clear History", variant="stop", size="sm")
    clear_btn.click(clear_history, outputs=[upload_status])
    gr.Markdown("---")
    gr.Markdown("### 📥 Export")
    pdf_btn = gr.Button("📄 Download PDF", size="sm", variant="primary")
    txt_btn = gr.Button("📋 Download TXT", size="sm", variant="primary")
    pdf_file = gr.File(label="PDF File", visible=True)
    txt_file = gr.File(label="TXT File", visible=True)
    export_status = gr.Textbox(label="Export Status", interactive=False, lines=2)
    pdf_btn.click(download_pdf, outputs=[export_status, pdf_file])
    txt_btn.click(download_txt, outputs=[export_status, txt_file])
    gr.Markdown("---")
    gr.Markdown("### 📱 Share")
    whatsapp_btn = gr.Button("💬 WhatsApp", size="sm", variant="secondary")
    facebook_btn = gr.Button("👍 Facebook", size="sm", variant="secondary")
    share_output = gr.Textbox(label="Share Link", interactive=False, lines=4)
    whatsapp_btn.click(generate_whatsapp_link, outputs=[share_output])
    facebook_btn.click(generate_facebook_link, outputs=[share_output])
```

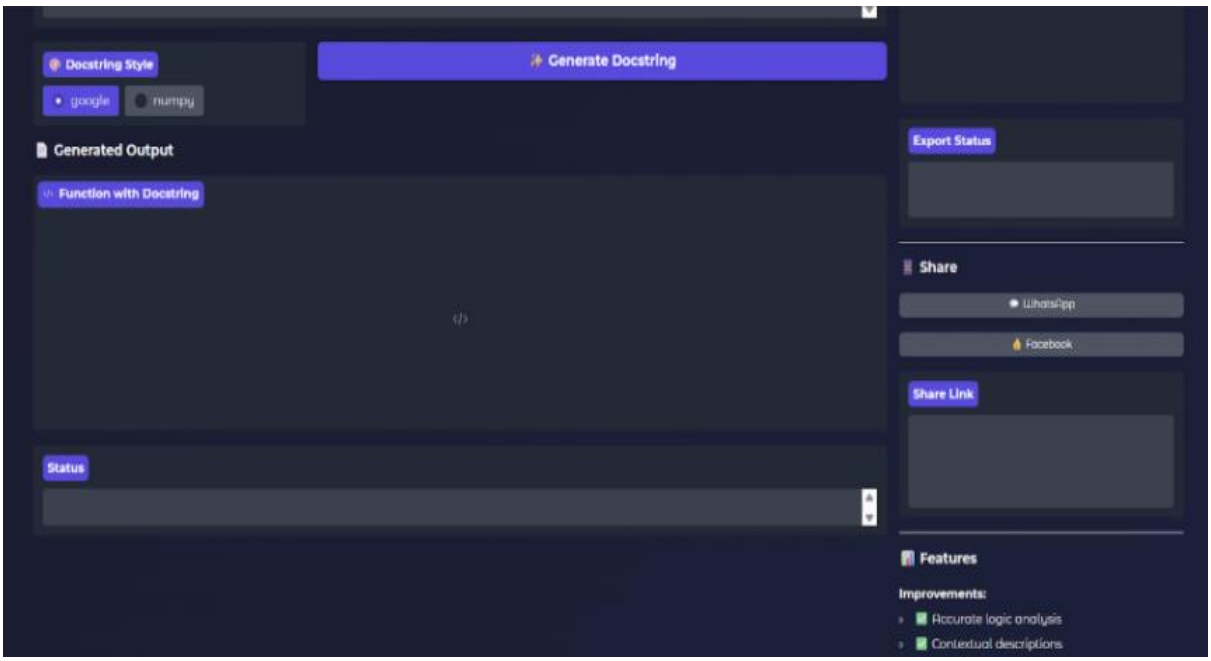
```
        gr.Markdown("---")
        gr.Markdown("""
        ### 📊 Features
        **Improvements:**
        - ✅ Accurate logic analysis
        - ✅ Contextual descriptions
        - ✅ Operation detection
        - ✅ Loop & condition recognition
        - ✅ Better PDF export
        - ✅ Consistent output
        ### 🎯 Supported Styles
        **Google:** Standard format
        **NumPy:** Scientific format
        """)
print("\n✅ Doc-Genie v2.0 Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

SCREENSHOTS

8.SCREENSHOTS

➢ Testing & Deployment



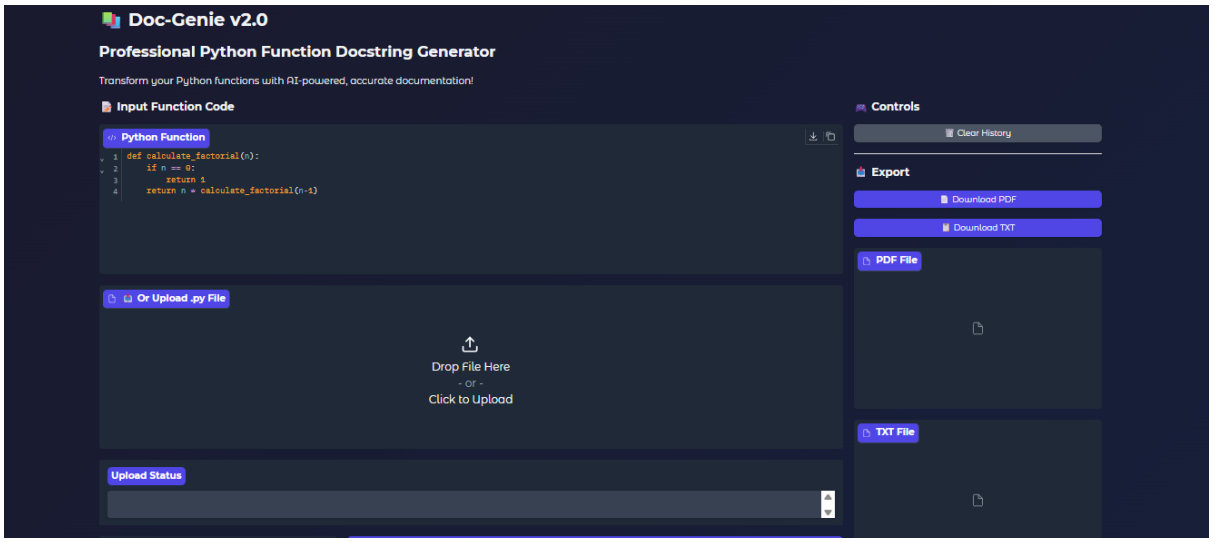


➢ Generated Gradio Public URL

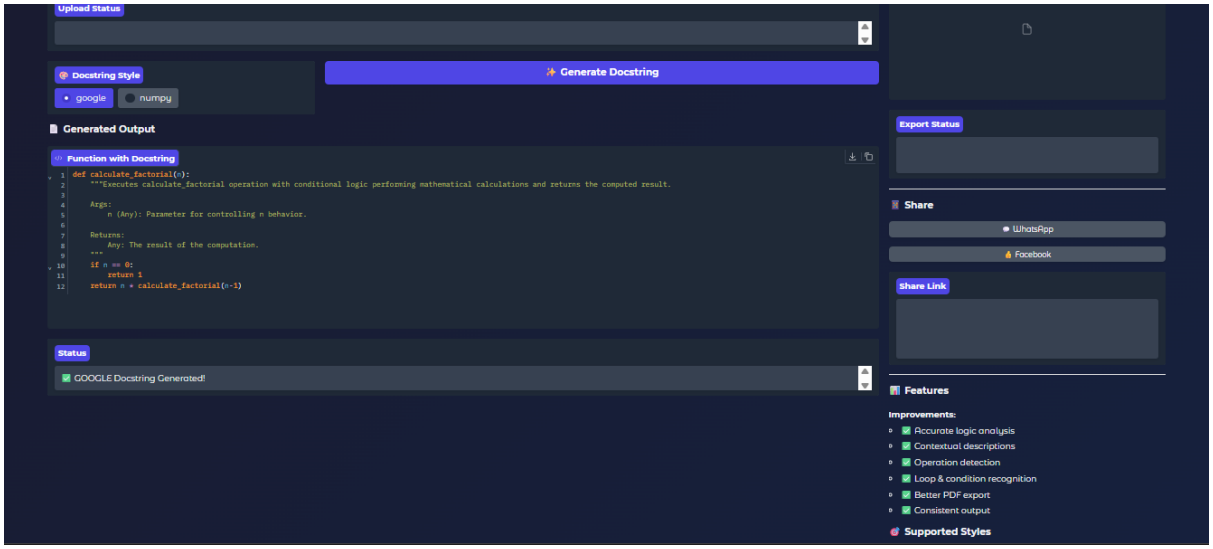
Public URL: `https://7f61586462fea23d34.gradio.live`

This screenshot shows the public URL generated by Gradio which allows users to access the application through a web browser.

➢ Website Generation Input Interface



➢ Generated Website Output



FUTURE ENHANCEMENT

8.FUTURE ENHANCEMENT

In future, the system can be improved by integrating more advanced **Artificial Intelligence models** to generate higher quality and more accurate documents. This will help the application automatically improve grammar, expand content, and provide better structured outputs for users.

Another enhancement is adding support for **multiple document formats and languages**. Currently the system may generate only limited formats, but future versions can support formats like PPT, HTML, and TXT, along with multi-language support such as Tamil, Hindi, and other regional languages.

The system can also be enhanced by implementing **user accounts and cloud storage integration**. This will allow users to save their generated documents online, access them later, and use the application easily through mobile devices with a dedicated mobile app.

CONCLUSION

9.CONCLUSION

The **Doc-Genie** system was developed to make document creation easier and faster by automatically generating documents based on user input. This project reduces the time and effort required for manual document preparation and helps users create well-structured documents efficiently.

Through this project, we implemented a simple and effective system that combines a user-friendly interface with backend processing to generate documents in a convenient way. The application demonstrates how technology can be used to automate repetitive tasks and improve productivity.

In conclusion, the Doc-Genie project successfully achieves its objective of providing an efficient document generation solution. With further enhancements such as advanced AI features, additional formats, and cloud integration, the system can be improved in the future to provide even more powerful and flexible document generation capabilities.